

Data Sharing

Mr. Tan Chiang Kang



Storing Data on Android

- We have learnt so far on techniques to store data in SQLite database, Shared Preferences and Files
- These solutions are all targeted at the private use of the applications or even the **Activities** themselves
- The created data are stored under local directories according to the package where the applications are created
- iOS has a lot of restrictions in accessing data belonging to other applications

Sharing Data

- Sometimes, we are interested to share the data with other applications that may use it, for example, an image taken by a camera app can be displayed by a gallery app



Content Providers

- The solution for sharing data between applications on Android is to use Content Providers
- Content Provider provides interface for publishing and consuming data, based around a simple URI addressing model using the `content://` schema
- They allow applications to browse for data stored on the file system, including external SD card locations
- Therefore all these data stored can be used by any application who wished to access them in the device

Content Providers

- Content Providers allow the data to be queried for results
- They also allow adding, updating and deleting of records
- The application can gain appropriate permission to add, remove or update data from any other application (including those from native Android databases)
- There are some native databases that are available as Content Providers to be accessed by third-party applications

Locating Content

- In order to access data made available by Content Providers, we need to point the location of where the resources are at
- We use URI as an address to refer to the resources
- For instance, if the resources are on the External Media Device (SD card) for **MediaStore**, the URI can be retrieved via **Media.EXTERNAL_CONTENT_URI**
- Another example is **CallLog** provider where call log entries can be accessed via **CallLog.Calls.CONTENT_URI**

Retrieve Data

- To retrieve data from Content Providers, we use a **managedQuery()** method
- The method returns a **Cursor** similar to the **query()** method previously discussed in SQLite
- The parameters of **managedQuery()** is similar by indicating the URI, contents to retrieve and other parameters to set conditions of WHERE and ORDER-BY clauses
- The returned **Cursor** can be read using similar ideas like in SQLite

Retrieve Data – example

```
String[] requestedColumns = {  
    MediaStore.Images.Media._ID,  
    MediaStore.Images.Media.TITLE  
};
```

```
Cursor cur = managedQuery(  
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI,  
    requestedColumns, null, null,  
    MediaStore.Images.Media.DATE_TAKEN + " ASC ");
```

Dealing with the Data

- The data extracted from the Content Provider can be used for display purposes with some potential options
- The data-binding to a Gallery widget makes it possible to show the information in a Gallery
- We can also choose a certain property of a record from the Content Provider to be stored
- The contents that the Content Provider refers to, such as an image, can be shown on the display by referring to the URI

Content Providers Permissions

- Some Content Providers require a special permission to access the information provided
- They are defined in the AndroidManifest.xml file using uses-permission tag
- Each of the Content Providers may have its own type of permissions
- Available permissions can be found in the class **android.Manifest.permission**

Native Content Providers

- Android has a number of native databases that are used as Content Providers
- Some example includes:
 - **Browser** – use the browser Content Provider to read or modify bookmarks, browser history, or web searches.
 - **CallLog** – view or update the call history (incoming and outgoing calls, missed calls and call details)
 - **ContactsContract** – used to retrieve, modify, or store your contact's details

Native Content Providers

- More examples:
 - **MediaStore** – provides centralised, managed access to multimedia on the device (audio, video, and images)
 - **Settings** – view most system settings and modify some of them
 - **UserDictionary** – access (or add to) user defined words added to the dictionary for use in IME predictive text input

MediaStore Content Provider

- The **MediaStore** Content Provider is used to access media on the phone
- It accesses audio, images, and video
- Each type of media is accessed through their respective Content Providers classes under **android.provider.MediaStore**
- We are allowed to retrieve, add, and delete media files from the device
- Some helper classes define the common data that can be requested

Common MediaStore Classes

- **Video.Media** – manages video files on the device
- **Images.Media** – manages image files on the device
- **Images.ThumbNails** – retrieves thumbnails for the images
- **Audio.Media** – manages audio files on the device
- **Audio.Albums** – manages audio files organized by the album they are a part of
- **Audio.Artists** – manages audio files by the artist who created them
- **Audio.Genres** – manages audio files belonging to a particular genre
- **Audio.Playlists** – manages audio files that are part of a particular playlist

MediaStore – example

```
String[] requestedColumns = {  
    MediaStore.Audio.Media.TITLE,  
    MediaStore.Audio.Media.DURATION  
};
```

```
Cursor cur = managedQuery(  
    MediaStore.Audio.Media.EXTERNAL_CONTENT_URI,  
    requestedColumns, null, null, null);
```

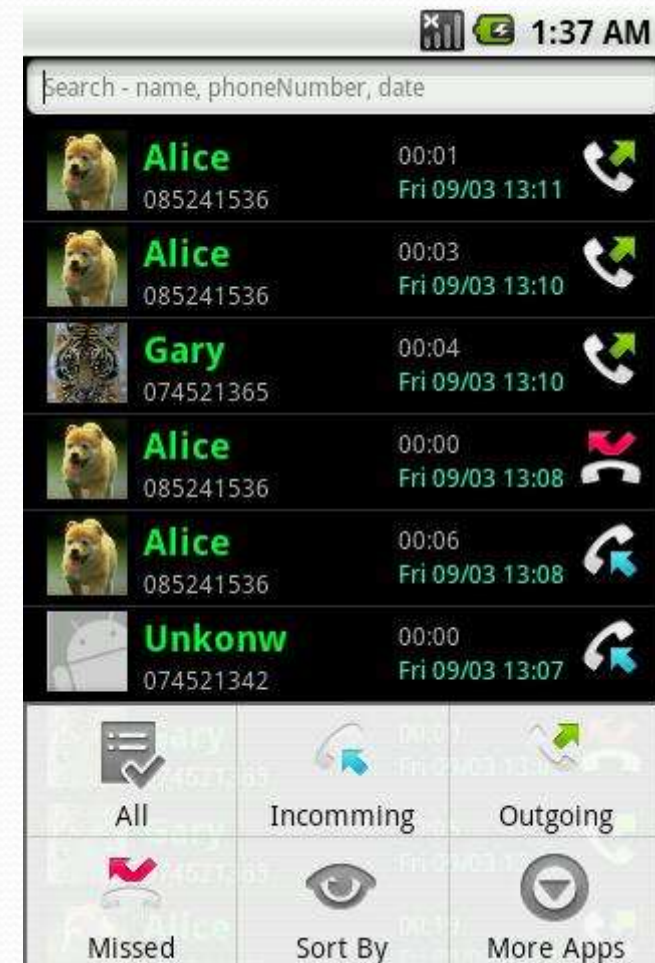
```
String[] columns = cur.getColumnNames();
```

MediaStore – example

```
int name =  
    cur.getColumnIndex(MediaStore.Audio.Media.TITLE);  
int size =  
    cur.getColumnIndex(MediaStore.Audio.Media.DURATION);  
  
cur.moveToFirst();  
while (!cur.isAfterLast()) {  
    Log.d("Debug Log", "Title" + cur.getString(name));  
    Log.d("Debug Log", "Length: " + cur.getInt(size) / 1  
        000 + " seconds");  
  
    cur.moveToNext();  
}
```

CallLog Content Provider

- The **CallLog** Content Provider provides access to the device's log history on recently dialed calls, received, and missed calls
- Import class **android.provider.CallLog** to achieve it
- It requires a permission of **READ_CONTACTS**



CallLog – example

```
String[] requestedColumns = {  
    CallLog.Calls.CACHED_NUMBER_LABEL,  
    CallLog.Calls.DURATION  
};
```

```
Cursor calls = managedQuery(  
    CallLog.Calls.CONTENT_URI, requestedColumns,  
    CallLog.Calls.CACHED_NUMBER_LABEL + " = ?",  
    new String[] { "ACertainPerson", "Jia Jun", },  
    null);
```

```
int durIdx =  
    calls.getColumnIndex(CallLog.Calls.DURATION);
```

CallLog – example

```
int totalDuration = 0;

calls.moveToFirst();
while (!calls.isAfterLast()) {
    Log.d("Debug Log", "Durations: " +
        calls.getInt(durIdx));
    totalDuration += calls.getInt(durIdx);

    calls.moveToNext();
}

Log.d("Debug Log", "ACertainPerson Total Call
    Duration: " + totalDuration);
```

Custom Content Provider

- To share other data that are not native, custom Content Provider can be created for the purpose
- Content Provider can be used to store the information on what are intended to be shared in SQLite database
- New class can be written that inherits the **ContentProvider** class (**android.content.ContentProvider**)
- The methods can be inherited with a skeleton code shown in the example:

Inheriting – example

```
public class CustomProvider extends ContentProvider {  
    public int delete(Uri uri, String selection, String[]  
        selectionArgs) {  
        return 0;  
    }  
  
    public String getType(Uri uri) {  
        return null;  
    }  
  
    public Uri insert(Uri uri, ContentValues values) {  
        return null;  
    }  
}
```

Inheriting – example

```
public boolean onCreate() {  
    return false;  
}  
  
public Cursor query(Uri uri, String[] projection, String  
    selection, String[] selectionArgs, String sortOrder)  
{  
    return null;  
}  
  
public int update(Uri uri, ContentValues values, String  
    selection, String[] selectionArgs) {  
    return 0;  
}  
}
```

Defining URI and Data Columns

- Other than the implemented methods, the custom Content Providers should also be defined with the URI and Data Columns
- The URI is defined in format as follows:

```
public static final Uri CONTENT_URI =  
    Uri.parse("content://my.edu.utar.CustomProvider");
```

- The data columns should be defined as well, acting such as fields in databases, for example:

```
public final static String _ID = "_id";  
public final static String _TITLE = "title";
```

Implementing Methods

- The methods should be implemented with details how they work in real
- As realised by now, the methods used in Content Providers resembles SQLite database methods
- Custom implemented methods can be based on SQLite databases which return the same data types, for example in **query()**

Updating the Manifest File

- The custom Content Provider should be included in the **AndroidManifest.xml** to exposed it to the rest of the system
- Example:

```
<provider
    android:authorities="my.edu.utar.CustomProvider"
    android:multiprocess="true"
    android:name="my.edu.utar.CustomProvider" >
</provider>
```

PIM

- PIM is Personal Information Management
- It was designed for Personal Digital Assistant (PDA) to use
- The main reason is for PDA to store, retrieve, and manage personal information
- Most PDA manufacturer installed its own proprietary application for this purpose
- Today, this is also extended to be used in smartphones and tablets that is filling and replacing the PDA's niche

PIM Databases

- PIM uses three types of databases
 - Contact Database
 - Contains names, addresses, phone numbers, and other similar information about personal contacts and follows the IETF vCard 2.1 specification
 - Event Database
 - An event is a single entry into the PIM event database. Fields used in the event database are subset of vCalendar 1.0
 - To Do Database
 - An activity to be done is a single entry into the PIM to do database. Fields used in the to do database are subset of vCalendar 1.0

vCard Overview

- vCard is the electronic business card.
- It is a powerful new means of Personal Data Interchange (PDI) that is automating the traditional business card.



**Here's my card, contact
information, website url and etc...**



vCard Overview

- Whether it's your computer (hand held organiser, Personal Information Manager (PIM), electronic email application, Web Browser) or telephone, the vCard revolutionises your personal communications
- For example, you can email the contact of someone in your Android smartphone to another smartphone user, the user will receive the PIM that contains all information of the person stored in your phone in one go (office, house, mobile phone number, fax, email and etc)



vCard Features

- vCards carry vital directory information such as name, addresses (business, home, mailing, parcel), telephone numbers (home, business, fax, pager, cellular, ISDN, voice, data, video), email addresses and Internet URLs (Universal Resource Locators)
- vCards can also have graphics and multimedia including photographs, company logos, audio clips (for name pronunciation) etc
- Geographic and time zone information in vCards allow others to know when to contact you



vCard Features

- vCards support multiple languages
- The vCard spec is transport and operating system independent so you can have vCard-ready software on any computer
- vCards are Internet friendly, standards based, and have wide industry support

vCard example

```
BEGIN:VCARD
VERSION:3.0
FN:Ms. Aaron Gates
N:Gates;Aaron;;;
NICKNAME:Aaron nickname
EMAIL;TYPE=INTERNET:aaron@example.com
EMAIL;TYPE=INTERNET:aaron_extra@example.com
TEL;TYPE=WORK:33333
TEL;TYPE=HOME:333331
TEL;TYPE=HOME;TYPE=FAX:333332
TEL;TYPE=PAGER:333333
TEL;TYPE=CELL:333334
ADR;WORK;POSTAL;CHARSET=UTF-8:PO Box;Extended Address;Work Street Address;Work Locality(Town);Work Region(State);Work Post Code;Work Country;Work
State;Work PostCode;Work Country
ADR;HOME;POSTAL;CHARSET=UTF-8:PO Box;Extended Address;Home Street Address;Home Locality(Town);Home Region(State);Home Post Code;Home Country;Work
City;Work State;Work PostCode;Work Country
LABEL;WORK;ENCODING=QUOTED-PRINTABLE:BusinessOffice=oD=oA=oD=oA, =oD=oA
LABEL;HOME;ENCODING=QUOTED-PRINTABLE:=oD=oA, =oD=oA
ORG:Example
TITLE:Engineer
BDAY:2009-01-01
PHOTO;ENCODING=BASE64;TYPE=GOOGLECONTACTSPHOTO_1140C7078B06ECD7:/9j/4A
AQSkZJRgABAQAAQABAAD/zwCEAAUDBAoOCREQDg8IDggICAoICAgOCAGlCQgICBAIBw
gICAglDRANCagOCQgIDBUNDhERExMTBwoWGBYSGBAXExlBBQUFCACHDwkjDxgVEhQYFh
kUFBQUFBQUFBQUFBwcfBQUBFBQUFBQUFRUWFBQUFBQUFBQUFB4UFBQeFBQeHv/AAB
EIAGAAAYAMBIgACEQEDEQH/xAAAdAAACAwACAwAAAAAAAAAAAAAHCAAFBgIDAQQJ/8QARh
AAAQIDBAUGCQgKAwAAAAAAGEDAAQSBQYREwchlJjzCDEoQIKyFEFRYWJxcqLCI4GRkq
GxwdEVJCUzNUNTgoPwY9Lh/8QAGgEAAwEBAQEAAAAAAAAAAAAAwQFBgECAP/EADQRAA
ECBAMFBgMJAAAAAAAAAAAEAgMEBRESITEyQUJxgSIoUZGhsRBP8QYUFSMzYYLh8P/aAA
wDAQACEQMRAD8AA+ji5YgLC1ONicu5UcjIkVKWhTsq66o6xkRWpFVFRahg96L35omUEX
nJeyWKxlpBsRIKXFJw2gMtZS+JFrXXA8vblpNGDIZcmwSBKy2YTgy7TmoYARa6VliX+6
DnYcmLUuACgiLbYoim7tJUW1Gbqk89rbNNrrKGbfMx3ZkNboArZmfEURDJBHdHsx6V5
2BnjdGZsc+TFoX/BjLLTNGoRPMb16klvrR2RIzgmIoNw4+ZTQe4aFZWcoaWQTDgJZ8qB
```

vCard example (cont)

utk2MyMw/my5EmpioeZTHnRF7MY6T5Oll4pXM2ih46GpfH3ggtxlhb1WZZo6/PNGbMxG
7ouL3JqmSMLbmbPRjMPlzXSF3KxXKzREcBdUMMUTx4xlr2aCrSYmsttvwvcsD8JllzGF
l+duokRaxXUqYQ2yp5h+qMeRXDmUk9kiDuw4yvxm7QBRWzzxqkevfo+n5lAKalplgJki
BgjEcHSb3xGnrJinPGaVpU14F9BJ9sfQY9eGO3Tu5ny4j7Inig+t16rblwmJU2HgYOVf
pzWklmmq8tRIPiWoqDAhFdUOs+oTOJh6FGbPjeF8+6PVgZPzjDwsaM7HQcFs6WLZlal
mnxwUkwQ+fnHnjCy3Jys/rz dqJ7Muov3pDsOtyztTbmEZs5DKzt5elucYPhhg2NoeCHd
GF8vjopzjB8MMGxujwQ7oxDq3D1WUkNt/NeY4vvCIqpKIgOoRLsil+lEmHhEVUlPbsai
JeqI9aKLRpYDlsT5kaqliSjg4tpsk694g9NotpVWEZKTdMPsNE/iOIMaLk6f7wV5YLqz
GCsA883VTnNjU2NPa1ooQ3Rmqccs/Z6oGGzpFtoEFsWwAdQgloA6vMnP64obTv7JMzSM
OvCD5btWyBLzUPmq8oaD8CgN2nHoVMSbGN/Md8kJpyScbLBwHBLskMdeHQo7PbeaUSQV
Ax5/H5lRYDd6bFKXeUVit7wF2h/wCoSKjSnSwxNN2+yDMSxh5jMKpiYRkb4Wk74bLsMI
g69MBmjiiJdqr5oPl57oywSpkIljrbaYHiW9q14eeBwaZFiQTFFrapeCwxcWHhQxiYx
lkTV5S93l6Y5xQ/CGBZ3R4QdoYXy8nTHOKHwwwbG6PBDuxarHD1UyQ23rL6U7RVqQLCL
c8sghX+mXWg9aC7t+CWMY3zmrea4dNJHmYmNXqEhT5oW7SumY+wxhjnkJb1PWHZ/8AYb
+xWqWATssNj9VMlqoOEGwcXirNNbijud4ABejfGoFalTJNRUoCXklvHC733swX5VxDsp
wRJ8XKflMwdrYLnHGDjpbL9Sw7TwQJHxxFU7QkMIimO4TLQDpZFqPadhOllr+SzeFyYs
hBdMiflHSbli3hb6gL6kSL7TgiBZpvU1FKJmUp4xXBD+iBdyRXFSbnwzsMwC9mlXB5ui
te7B5vjZ+dIvNquGdLOtiU44VIqIWHmi/hbFlyHZgosoTFkh42l8kFdGuj51+omLRNWk
k/Bqzm+c1qRaKk8WCwbrxyROSpGNNZjSmOoYFXJzvortUirNP6JYFrwnMqz6VUaqMNjH
1wWLwzytS5mg1K2NVFVNxzx9BhwhAs3Zt6LiJNh/AxN36896D9tXYfZCpwWoCrCoSqik
jY3qvgj8vRRSW8W1VTGQjFzrITlloRuElFDA7sHJL3eXpjnED4YYFndHgh3YXy8nTHOK
HwwwbG6PBDujD9Y4eqjyG29YPSB/FJP/esMOFI7g8MPuhO9l5oNpShLVQO8VO7tD73mh
wrOPFoV/4gX7MYtUbuwVuk7cTmPZZXS5oL/KP4wKh3k9qCrpd6F/ICBQpYa+ztRDrneu
gRJ79RdPJQ/ik/xPiWGLmdxfZX7oXlklnjaU+vacEveKGJnywbJfBL9ixp5bKXRKX3X
z9ou/JpT9v2h7XxFB3vqn6k7w/ygC8ml8ben17Wv3lg9X16E7w/wAoGo3lDyPsvNP7sf
5e6B8SIsSMikUvV4+mOcUPwhgmdoeEHdGF9vLopzih+EMExujwQ7oxaq/D1UuQ23rD6Z
2F8DEoUkJt8RHDq66qvRgcNTceef2QZcFcRclmtrokGkvtgAXks7OlzbsicbIWYXdFzt
FGm5Kt7hKVKScIkmpAyyhLZJinEsSES5hRf6UPoKOC3ATmFYkXiHMkHiHqP6RLomyqn
IlhziYH9WAXpuoLRrRb1/7RhiZtgTBUXWBjSvslC/aSrznHWrEqyrgvbjTdqizooXVI
S3tXPHatlvix2vaMjkU1UlbtpovyXPkgSZL4W//Km3gytXZrq7oGy+E5lypr5qdfpaop
tENzUs+zQYqqcFSddc8rjmsk9lOZlrdLNRjssovWzHfZ6vzxQmYn3eUN9bZcyiQGGXIQ
12tvUoX8mBP25P/wC9ZYPV9ehOcNYXXQfP+C3mdbcUU/SYkTZLq3cSERXtLzQzU/Ki4o

vCard example (cont)

QluuDSUdlx8SVsN4+SFTOTLloiuUAVFflXzjHiNbaujddqWZdezpx2moG3HvkA1puN4c
/zxxoxo5JulnhrzmRfJKvhuYbOS83k6Y5xQ+GGDZ3R4QdoYXy8nTHOKHwwwbG6PBDujF
CscPVSpDbeucZ22rEcSaCaliFufYISqX92+I/y3BH6loUSJMGM6E7E3VUCwEWKitotMr
amDU82crNOqLbaqVTUw4Wzioqbo4+Wc2KwoulWzzKXFisan5RwSDtCONRFtdVIZHRZeV
ucsipisqvkhad8zzalLnvJG1pk6ZiHd2qoyMy57zDedNP3XRfe9+QStgBK7hVruDj+M
CuZflyVSUiMtoiXrQZroXaamMFOTCHritJknkKA9akugPuAlSiy8QCS7yiPa9KIdbZGD
7uPZ3fRcnQ+gzpuVNalkNukikhI63+6eHZcaLtAXaTxRrLBvbNMgg5pvUjTmO63C9Iqe
tFHEiTDm4oMWY4gJJhLDduRVnatuvO6nDJQ/p9X2orlkd8hLkbogOtTLBB8sDc98V2ZJ
JXSS45r//Z
URL;TYPE=WORK:http://qt.nokia.com/logo.png
URL;TYPE=HOME:http://qt.nokia.com/logo.png
NOTE:Some notes are here
END:VCARD

BEGIN:VCARD
VERSION:3.0
FN:Mr. Alexander Mcdonald
N:Mcdonald;Alexander;;;
EMAIL;TYPE=INTERNET:alex@example.com
TEL;TYPE=WORK:1111111111
ADR;TYPE=WORK;POSTAL;CHARSET=UTF-8:PO Box;Extended Address;Work Street Address;Work Locality(Town);Work Region(State);Work Post Code;Work Country;Work
City;Work State;Work PostCode;Work Country
ADR;TYPE=HOME;POSTAL;CHARSET=UTF-8:PO Box;Extended Address;Home Street Address;Home Locality(Town);Home Region(State);Home Post Code;Home Country;Work
City;Work State;Work PostCode;Work Country
ORG:Example
TITLE:Engineer
BDAY:2010-02-02
END:VCARD



vCalendar Overview

- vCalendar defines a transport and platform-independent format for exchanging calendaring and scheduling information in an easy, automated, and consistent manner.
- It captures information about events and "to-do" items that are normally used by applications such as personal information managers (PIMs) and group schedulers.
- Programs that use vCalendar can exchange important data about events so that you can schedule meetings with anyone who has a vCalendar-aware program.

vCalendar Usage

- Using Email to Schedule a Meeting
 - You need to schedule a meeting with fellow employees from several divisions, customer representatives and key vendors. The people you want to invite use a variety of scheduling applications or different operating systems.
 - You broadcast an e-mail with a vCalendar attachment to all the invitees. Each receives the e-mail, selects the vCalendar attachment, drags the icon and drops it onto their particular scheduling application. Instantly, all of the meeting information-from date and time to location and related events-is easily and automatically added to the invitees' calendar.

vCalendar Usage

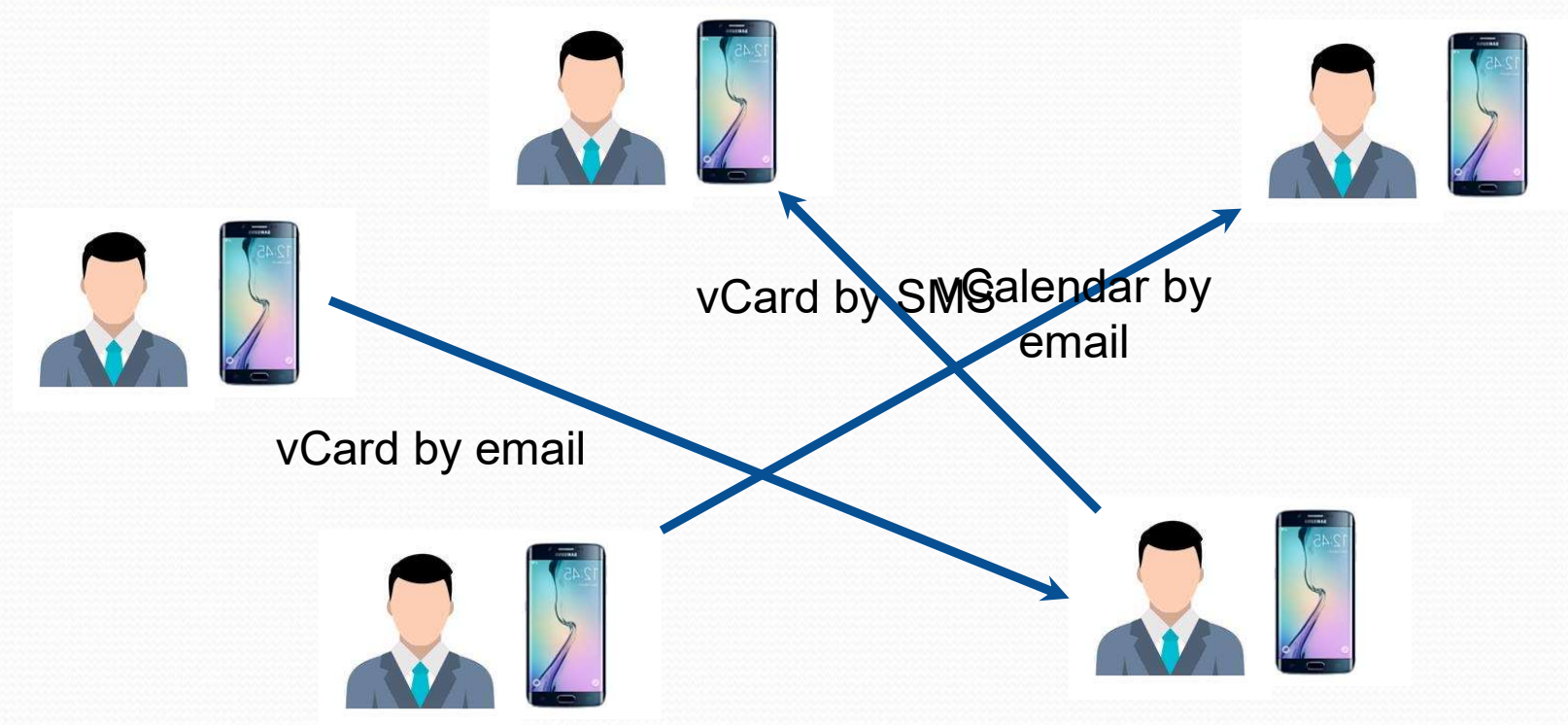
- Event Planning

- While surfing on the Internet you come across a website with an interesting trade show. The website has a calendar of events including information on trade show registration, travel planning, deadlines for booking either floor space or local hotel accommodations, etc.
- You click on the associated vCalendar icon and, in a flash, receive a sequence of calendar events, as well as to-do or action items required to register or prepare for the show as either a participant or an exhibitor. Automatically, the information, in a flexible vCalendar format, is easily integrated into your calendaring and scheduling application. You email the vCalendar object to colleagues who may also have an interest in this event.

Android & PIM

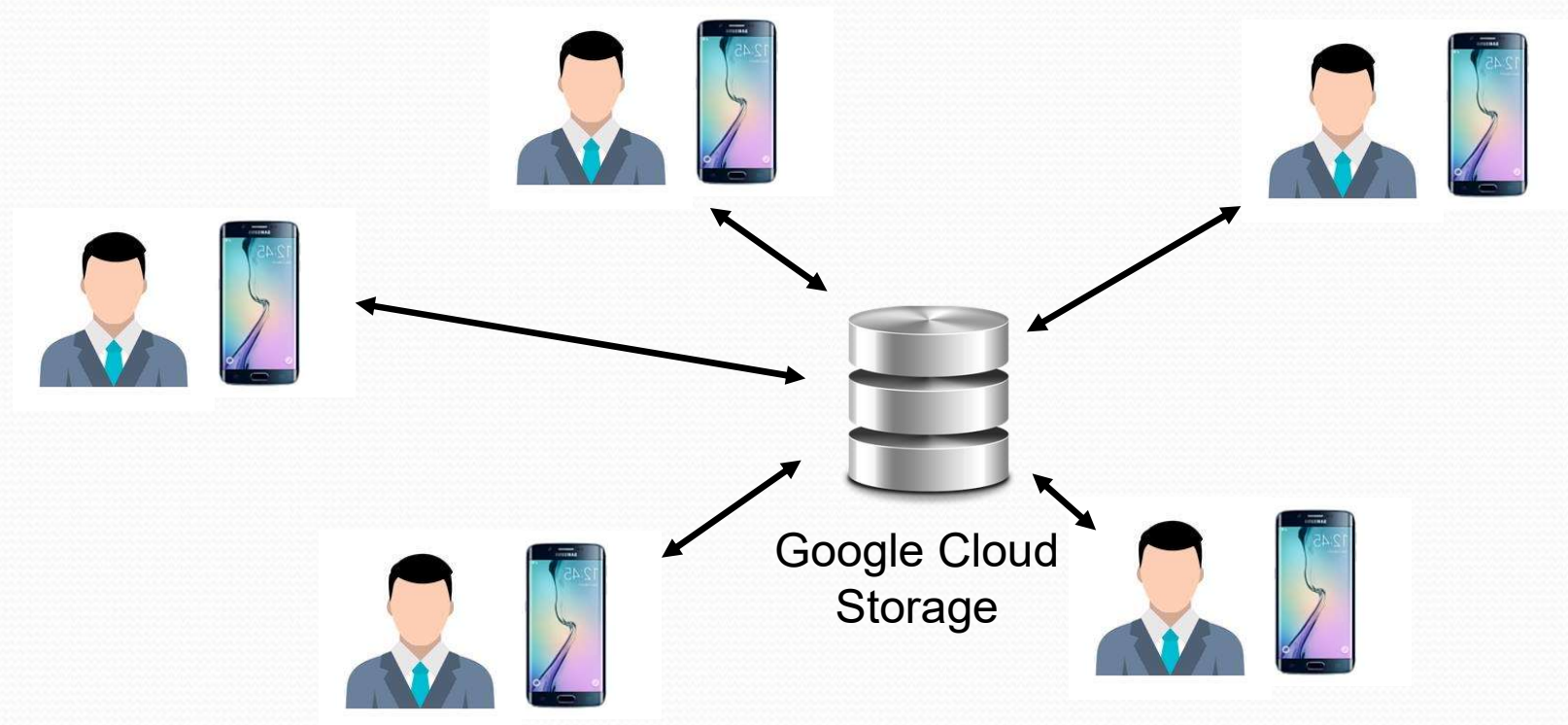
- Although smartphones like Android do allow PIM to be used, but Android had yet to be supporting PIM much by providing API for it (as in Java MIDP PIM API)
- Android uses Google technology to store contacts and calendars which made it easier to retrieve on a different machine
 - Do you know how Google's way of storing your contacts and calendars is different?
- It is also easier to recover the contacts and calendars if a certain Android device is stolen or spoilt

Android & PIM



- Peer-to-peer data exchange
- Local data storage

Android & PIM



- Centralised data storage
- Client-server data synchronisation



What's Next?

- Android networking



Introduction to Networking

- Mobile devices are more powerful with the extended capability to communicate
- Other than standard GSM communications (making and receiving calls/SMS), the current mobile devices allow Internet data access
- Currently, most mobile devices are ready with network connection through 3G and above (although some basic phone may still work on GPRS)
- Even without mobile Internet, some devices are capable of connecting through Wi-Fi

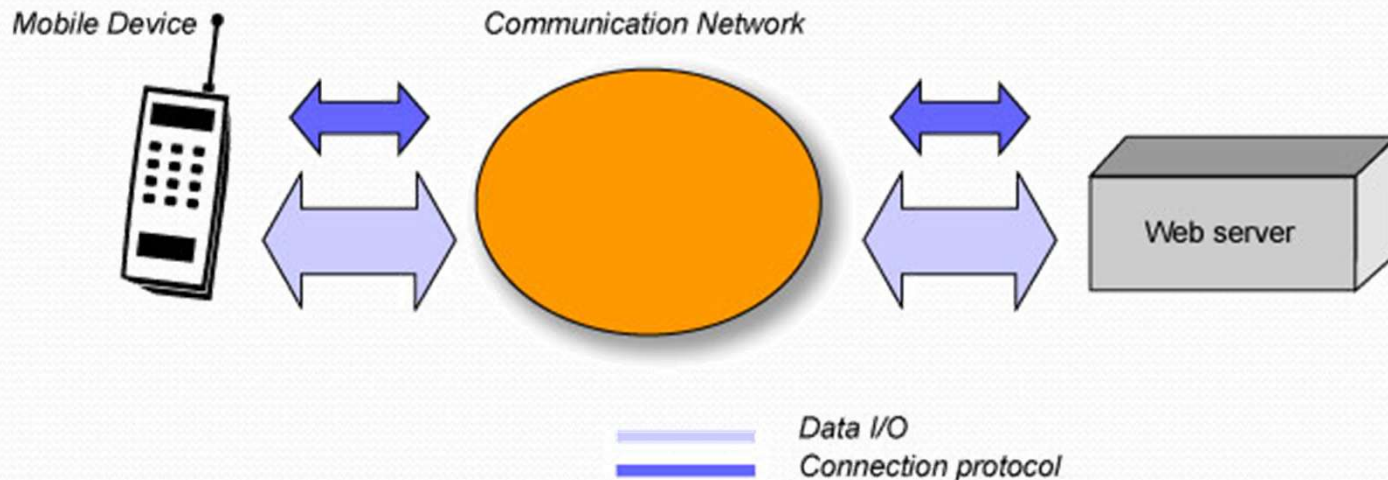


Connecting to the Internet

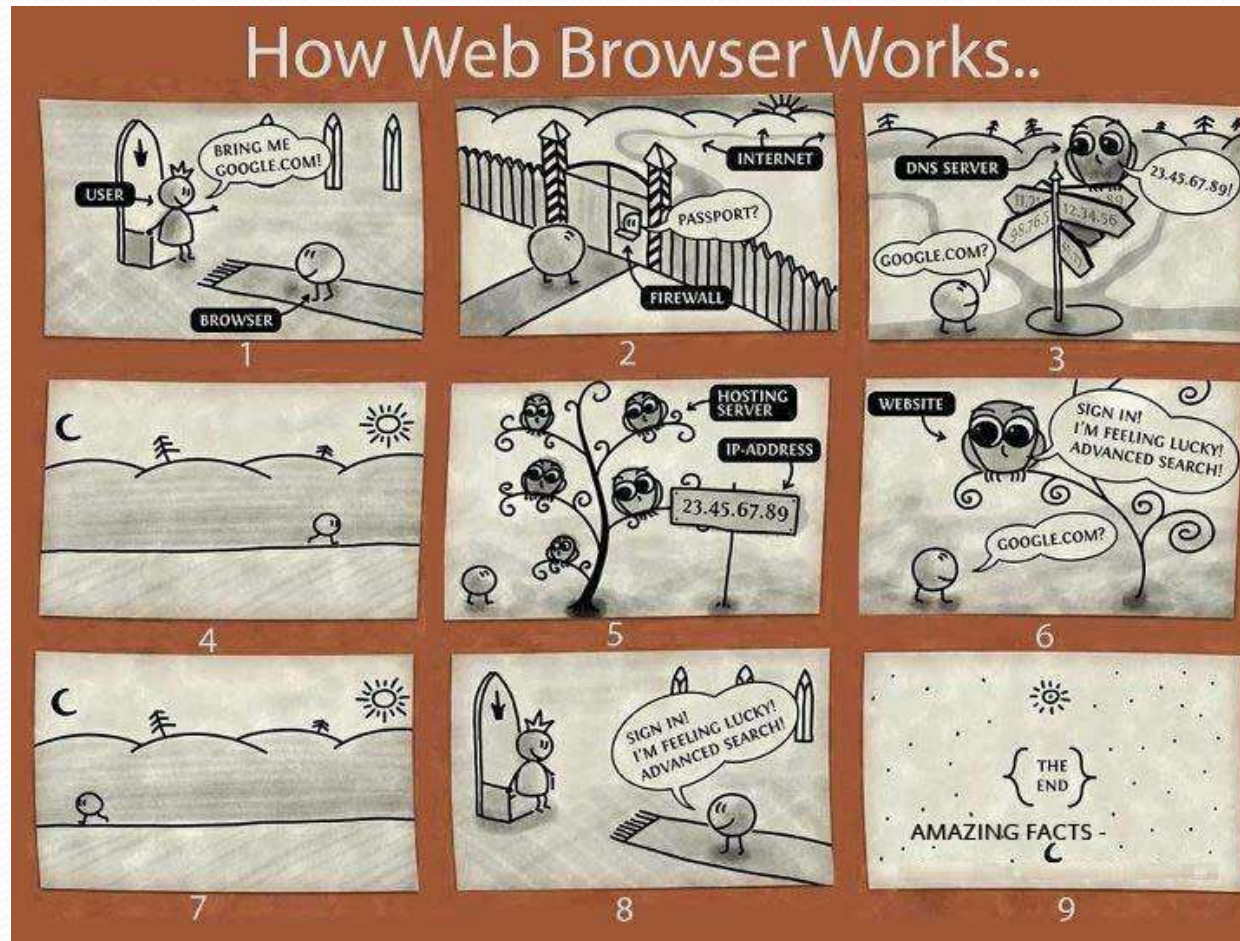
- Connection to the Internet is an important feature for mobile applications
- First of all, connecting to the Internet allows the user to access websites along with other resources stored on the Internet
- Other than that, the Internet also provides web services (different from websites)

HTTP Networking

- The communication between a mobile device and a Web server is based on HTTP (Hypertext Transfer Protocol)
- HTTP is a connection-oriented request-response application layer protocol



The Idea of World Wide Web



- Source: From Facebook

Android Networking

- Android provides networking capability through the packages in **java.net**
- Accessing the Internet through HTTP is the commonest way to transfer data to and from the network
- The java.io package provides input/output (I/O) capability that is used in handling the transferred data

Setting Permission

- For Android applications, permission is required for networking to work
- The permission can be added into the AndroidManifest.xml
- The following statements should be added to the manifest file:

```
<uses-permission  
    android:name="android.permission.INTERNET"/>
```

Data Reading

- **InputStream** is used to process the data input when the data are transferred to the application (similar to using reading data from a file)
- However reading data from the network is easily prone to errors
- Errors can occur due to poor network coverage, server down, invalid URL etc
- These need to be carefully taken care of to avoid problems to your application

Data Reading

- Use **URL** object to connect to a targeted url
- Read the return data from the established connection (binary)
- Interpret the data that is packaged back into its original format (e.g. HTML, images, files and etc)

Using HttpURLConnection

- **HttpURLConnection** object is used to open a HTTP connection to a targeted URL.
- It is helpful to extract information regarding the URL before real data are being transferred
- These information include the HTTP status and header information
- For example, we can retrieve the length of the content, content type, and date-time information
- Knowing some of these properties will help in correct reading of the data
- Import **java.net.HttpURLConnection**

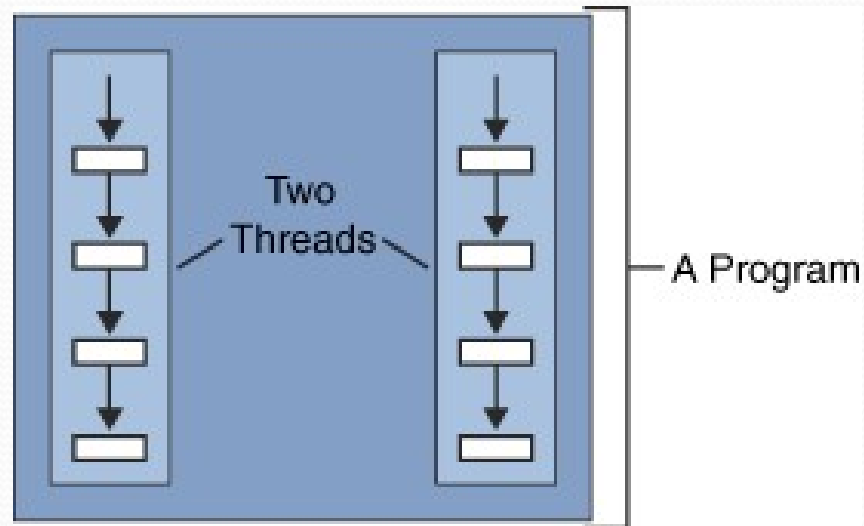
XML Pull Parser

- There are a good number of data that are in the form of XML on the Internet (e.g. RSS feeds)
- Android offers a variety of XML utilities, here we can make use of XML Pull Parser
- The XML can then easily be interpreted according to tag and other attributes in within
- Import **`org.xmlpull.v1.XmlPullParser`** and **`org.xmlpull.v1.XmlPullParserFactory`**

Using Threads

- Networking processes can be rather time consuming, depending on delivered data size and network speed
- When the application is running networking, it will cause UI to be blocked until the operation is completed
- To avoid such problem, the networking operations should be done on a separated thread

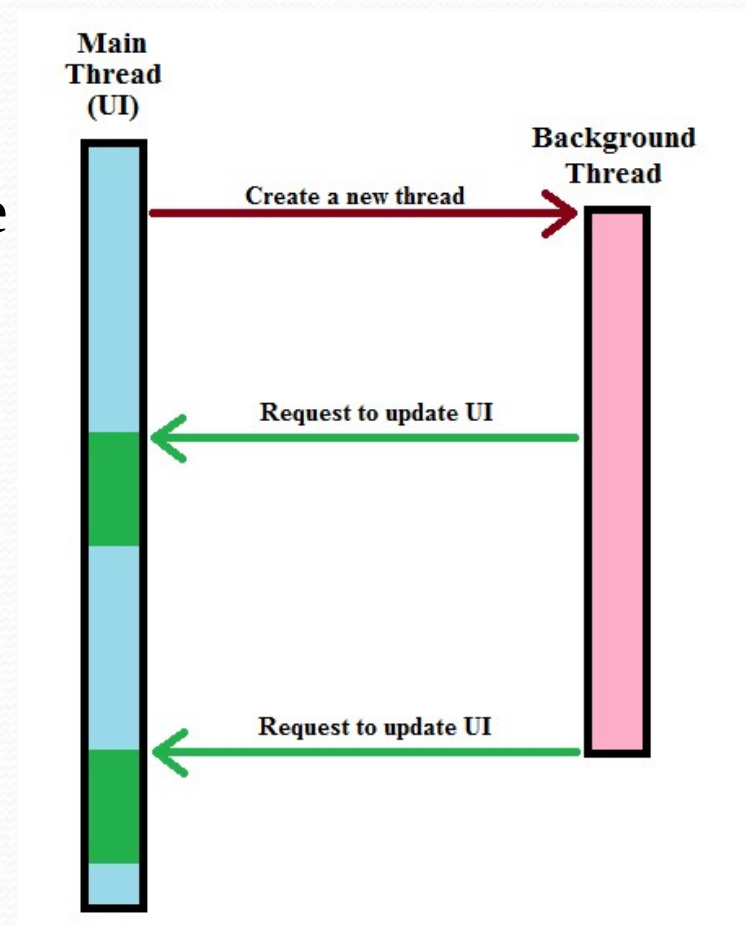
Creating New Thread



- To create a new **Thread**, a **run()** method needs to be defined
- The **run()** method defines what needs to be done in the separate thread, in this case the connecting and transfer of data that are time consuming

Between Threads

- Example scenario:
 - A main thread can be running and wants to execute some tasks in the background thread
 - It creates a new thread
 - The thread runs in the background
 - Occasionally the background thread may post updates to the main thread to update UI



Using Handlers

- In Android, handling multiple threads can be challenging
- The working Thread cannot update on the main (UI) Thread automatically
- **Handler** can be used on the UI Thread to receive data posted back to the UI thread to alter the UI
- Import **android.os.Handler** class

AsyncTask

- Due to the challenges of passing data through and forth between Threads, Android introduced **AsyncTask** to help ease things up
- The **AsyncTask** is introduced in Android 1.5 in the **android.os** package
- It is an abstract helper class for managing background operations that will eventually post back to the UI thread
- Developers can create subclasses of **AsyncTask** to do background processes rather than creating Threads.

AsyncTask

- In order to use AsyncTask, the **doInBackground()** method has to be overridden.
- Some common methods:
 - **doInBackground()** – processes that are to be running on the background thread.
 - **onPreExecute()** – called on the UI thread before the thread starts running.
 - **onProgressUpdate()** – called when **publishProgress()** is invoked in the **doInBackground()**.
 - **onPostExecute()** – called on the UI thread after the background thread finishes.

AsyncTask example

```
public class AsyncTaskTestActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
  
        new PostTask().execute("http://www.trytofetchdata.com.my");  
    }  
  
    // The definition of the AsyncTask  
    private class PostTask extends AsyncTask<String, Integer, String> {  
        @Override  
        protected void onPreExecute() {  
            super.onPreExecute();  
            displayProgressBar("Downloading...");  
        }  
    }  
}
```

AsyncTask example

```
@Override
protected String doInBackground(String... params) {
    String url=params[0];

    // Execute something
    for (int i = 0; i <= 100; i += 5) {
        publishProgress(i);
    }
    return "All Done!";
}
```

```
@Override
protected void onProgressUpdate(Integer... values) {
    super.onProgressUpdate(values);
    updateProgressBar(values[0]);
}
```

AsyncTask example

```
@Override
protected void onPostExecute(String result) {
    super.onPostExecute(result);
    dismissProgressBar();
}
}
```



Mobile Device and Web Content

- Let's take a break from Mobile Networking for a moment
- What can we achieve by reading Internet contents?
- Remember we can also access web services?
- What are web services and how can they be created?
- Let's park the mobile device centric topics for a while and take a look at web backend before we return...



What's Next?

- Introduction to Web Backend